

13.0 DIRECT MEMORY ACCESS CONTROLLER

The ENC28J60 incorporates a dual purpose DMA controller which can be used to copy data between locations within the 8-Kbyte memory buffer. It can also be used to calculate a 16-bit checksum which is compatible with various industry standard protocols, including TCP and IP.

When a DMA operation begins, the EDMAST register pair is copied into an Internal Source Pointer. The DMA will execute on one byte at a time and then increment the Internal Source Pointer. However, if a byte is processed and the Internal Source Pointer is equal to the Receive Buffer End Pointer, ERXND, the Source Pointer will not be incremented. Instead, the Internal Source Pointer will be loaded with the Receive Buffer Start Pointer, ERXST. In this way, the DMA will follow the circular FIFO structure of the receive buffer and received packets can be processed using one operation. The DMA operation will end when the Internal Source Pointer matches the EDMAND Pointer.

While any DMA operation is in progress, the DMA Pointers and the ECON1.CSUMEN bit should not be modified. The DMA operation can be canceled at any time by clearing the ECON1.DMAST bit. No registers will change; however, some memory bytes may already have been copied if a DMA copy was in progress.

Note 1: If the EDMAND Pointer cannot be reached because of the receive buffer wrapping behavior, the DMA operation will never end.

2: By design, the DMA module cannot be used to copy only one byte (EDMAST = EDMAND). An attempt to do so will overwrite all memory in the buffer and may never end.

13.1 Copying Memory

To copy memory within the buffer:

1. Appropriately program the EDMAST, EDMAND and EDMADST register pairs. The EDMAST registers should point to the first byte to copy from, the EDMAND registers should point to the last byte to copy and the EDMADST registers should point to the first byte in the destination range. The destination range will always be linear, never wrapping at any values except from 8191 to 0 (the 8-Kbyte memory boundary). Extreme care should be taken when programming the Start and End Pointers to prevent a never ending DMA operation which would overwrite the entire 8-Kbyte buffer.
2. If an interrupt at the end of the copy process is desired, set EIE.DMAIE and EIE.INTIE and clear EIR.DMAIF.
3. Verify that ECON1.CSUMEN is clear.
4. Start the DMA copy by setting ECON1.DMAST.

If a transmit operation is in progress (TXRTS set) while the DMAST bit is set, the ENC28J60 will wait until the transmit operation is complete before attempting to do the DMA copy. This possible delay is required because the DMA and transmission engine share the same memory access port.

When the copy is complete, the DMA hardware will clear the DMAST bit, set the DMAIF bit and generate an interrupt (if enabled). The pointers and the EDMACS registers will not be modified.

After the DMA module has been initialized and has begun its copy, two main clock cycles will be required for each byte copied. As a result, if a maximum size 1518-byte packet was copied, the DMA module would require slightly more than 121.44 μ s to complete. The time required to copy a minimum size packet of 64 bytes would be dominated by the time required to configure the DMA.

13.2 Checksum Calculations

The checksum calculation logic treats the source data as a series of 16-bit big-endian integers. If the source range contains an odd number of bytes, a padding byte of 00h is effectively added to the end of the series for purposes of calculating the checksum. The calculated checksum is the 16-bit one's complement of the one's complement sum of all 16-bit integers. For example, if the bytes included in the checksum were {89h, ABh, CDh}, the checksum would begin by computing 89ABh + CD00h. A carry out of the 16th bit would occur in the example, so in 16-bit one's complement arithmetic, it would be added back to the first bit. The resulting value of 56ACh would finally be complemented to achieve a checksum of A953h.

To calculate a checksum:

1. Program the EDMAST and EDMAND register pairs to point to the first and last bytes of buffer data to be included in the checksum. Care should be taken when programming these pointers to prevent a never ending checksum calculation due to receive buffer wrapping.
2. To generate an optional interrupt when the checksum calculation is done, clear EIR.DMAIF, set EIE.DMAIE and set EIE.INTIE.
3. Start the calculation by setting ECON1.CSUMEN and ECON1.DMAST.

When the checksum is finished being calculated, the hardware will clear the DMAST bit, set the DMAIF bit and an interrupt will be generated if enabled. The DMA

Pointers will not be modified and no memory will be written to. The EDMACSH and EDMACSL registers will contain the calculated checksum. The host controller may write this value into a packet, compare this value with a received checksum, or use it for other purposes.

Various protocols, such as TCP and IP, have a checksum field inside a range of data which the checksum covers. If such a packet is received and the host controller needs to validate the checksum, it can do the following:

1. Read the checksum from the packet and save it to a temporary location
2. Write zeros to the checksum field.
3. Calculate a new checksum using the DMA controller.
4. Compare the results with the saved checksum from step 1.

Writing to the receive buffer is permitted when the write address is protected by means of the ERXRDP T Pointers. See **Section 7.2 "Receiving Packets"** for additional information.

The IP checksum has unique mathematical properties which may be used in some cases to reduce the processing requirements further. Writing to the receive buffer may be unnecessary in some applications.

When operating the DMA in Checksum mode, it will take one main clock cycle for every byte included in the checksum. As a result, if a checksum over 1446 bytes were performed, the DMA module would require slightly more than 57.84 μ s to complete the operation.

TABLE 13-1: SUMMARY OF REGISTERS ASSOCIATED WITH THE DMA CONTROLLER

Register Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Reset Values on page
EIE	INTIE	PKTIE	DMAIE	LINKIE	TXIE	r	TXERIE	RXERIE	13
EIR	—	PKTIF	DMAIF	LINKIF	TXIF	r	TXERIF	RXERIF	13
ECON1	TXRST	RXRST	DMAST	CSUMEN	TXRTS	RXEN	BSEL1	BSEL0	13
ERXNDL	RX End Low Byte (ERXND<7:0>)								13
ERXNDH	—	—	—	RX End High Byte (ERXND<12:8>)					13
EDMASTL	DMA Start Low Byte (EDMAST<7:0>)								13
EDMASTH	—	—	—	DMA Start High Byte (EDMAST<12:8>)					13
EDMANDL	DMA End Low Byte (EDMAND<7:0>)								13
EDMANDH	—	—	—	DMA End High Byte (EDMAND<12:8>)					13
EDMADSTL	DMA Destination Low Byte (EDMADST<7:0>)								13
EDMADSTH	—	—	—	DMA Destination High Byte (EDMADST<12:8>)					13
EDMACSL	DMA Checksum Low Byte (EDMACS<7:0>)								13
EDMACSH	DMA Checksum High Byte (EDMACS<15:8>)								13

Legend: — = unimplemented, read as '0'. Shaded cells are not used with the DMA controller.